Matrix Multiplication as Composition

Contents

1. Overview 2
2. Composition: one transformation after another 3
   - 2.1 Turning a stage twice 3
   - 2.2 Following the basis vectors 5
3. Why the product is written right to left 6
4. Computing a product column by column 7
5. Order matters 10
6. Grouping does not matter 12
7. Where ML multiplies matrices 12
   - 7.1 The shape rule 12
   - 7.2 Points as rows: the same product, transposed 13
   - 7.3 Linear layers collapse into one 14
8. Summary 16
9. Sources 16
10. Key terms 17

Where this fits: Pipeline map step 0 (Foundations). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	8 Model	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Dot product; Linear transformations and matrices.
- **Leads to:** Jacobian and matrix gradients; Neural networks; Forward propagation.

1. Overview

Key point: Multiplying two matrices gives the matrix of "apply one transformation, then the other"; we read the **product** (G-1179) from right to left.

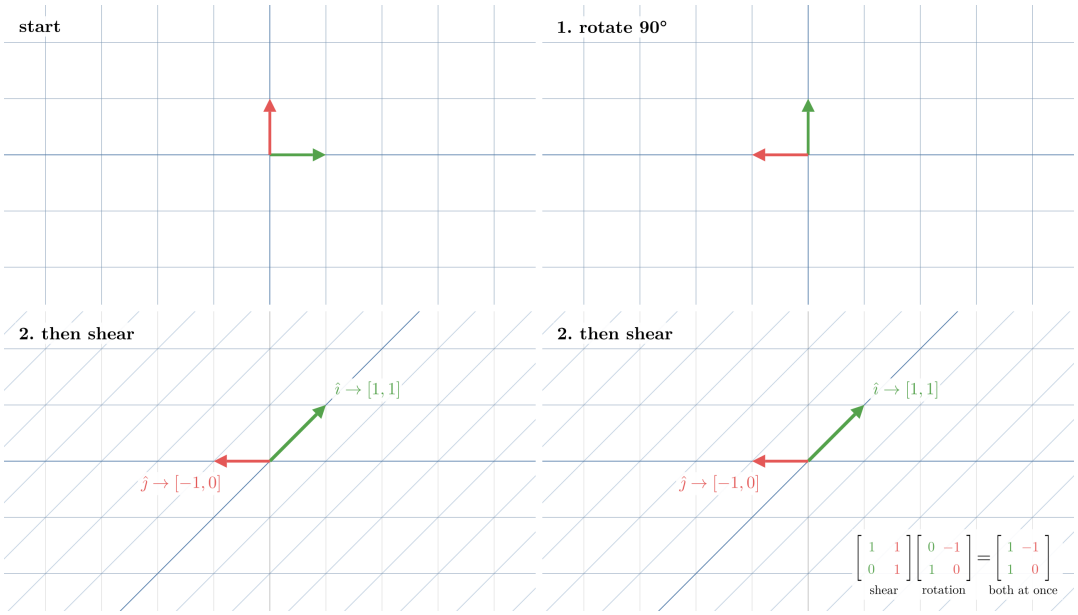


Figure 1: Rotating the plane by 90° and then shearing it is one single transformation; its matrix is the product of the two matrices

Figure 1 applies two transformations one after the other: a rotation by 90°, then a shear. From start to finish, the plane has undergone a third linear transformation, with a matrix of its own. That matrix is the product of the shear matrix and the rotation matrix.

This Note builds on the matrix of a transformation: the columns of a matrix are where $\hat{\imath} = [1, 0]$ and $\hat{\jmath} = [0, 1]$ (the two unit arrows along the axes, the **basis vectors**) land, and $A\mathbf{x}$ is the linear combination of those columns (a sum of the columns, each multiplied by a number, see **linear combinations** (G-1091)).

2. Composition: one transformation after another

> **Key point:** Applying one linear transformation and then another is again a linear transformation, called their **composition** (G-431); following $\hat{\imath}$ and $\hat{\jmath}$ gives its matrix.

2.1 Turning a stage twice

> **Key point:** Two turns of a stage, one after the other, are the same as one combined turn; the matrix of that combined turn is the **product** (G-1179) of the two turn matrices.

Picture a concert stage with x and y axes centred on it. A seat sits at $(2, 1)$. Now the stage turns, and every seat turns with it (Figure 2):

1. **First turn, 180°.** The seat moves to $(-2, -1)$.
2. **Second turn, 90° clockwise.** The seat moves to $(-1, 2)$.

Each turn is a linear transformation (see what makes a transformation linear), so each has a matrix whose columns are where $\hat{\imath}$ and $\hat{\jmath}$ land. The 180° turn sends $\hat{\imath}$ to $[-1, 0]$ and $\hat{\jmath}$ to $[0, -1]$; the 90° clockwise turn sends $\hat{\imath}$ to $[0, -1]$ and $\hat{\jmath}$ to $[1, 0]$:

$$T_1 = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}$$

$$T_2 = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

Done the long way, the seat needs two multiplications. Each entry of the result is two products added, one entry per line.

First turn, $T_1[2, 1]$:

$$\text{top} = (-1)(2) + (0)(1) = -2 + 0 = -2$$

$$\text{bottom} = (0)(2) + (-1)(1) = 0 - 1 = -1$$

Second turn, $T_2[-2, -1]$:

$$\text{top} = (0)(-2) + (1)(-1) = 0 - 1 = -1$$

$$\text{bottom} = (-1)(-2) + (0)(-1) = 2 + 0 = 2$$

So the seat goes $[2, 1] \to [-2, -1] \to [-1, 2]$. Done the short way, we first combine the two turns into one matrix, the product T_2T_1. Each entry is a row of T_2 times a column of T_1: the first entry of the row times the first entry of the column, plus the second times the second.

$$\text{row 1, column 1} = (0)(-1) + (1)(0) = 0 + 0 = 0$$

$$\text{row 1, column 2} = (0)(0) + (1)(-1) = 0 - 1 = -1$$

$$\text{row 2, column 1} = (-1)(-1) + (0)(0) = 1 + 0 = 1$$

$$\text{row 2, column 2} = (-1)(0) + (0)(-1) = 0 + 0 = 0$$

$$T_2T_1 = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

Applied to the seat:

$$(0)(2) + (-1)(1) = -1$$

$$(1)(2) + (0)(1) = 2$$

so $T_2T_1[2, 1] = [-1, 2]$, the same spot in one step.

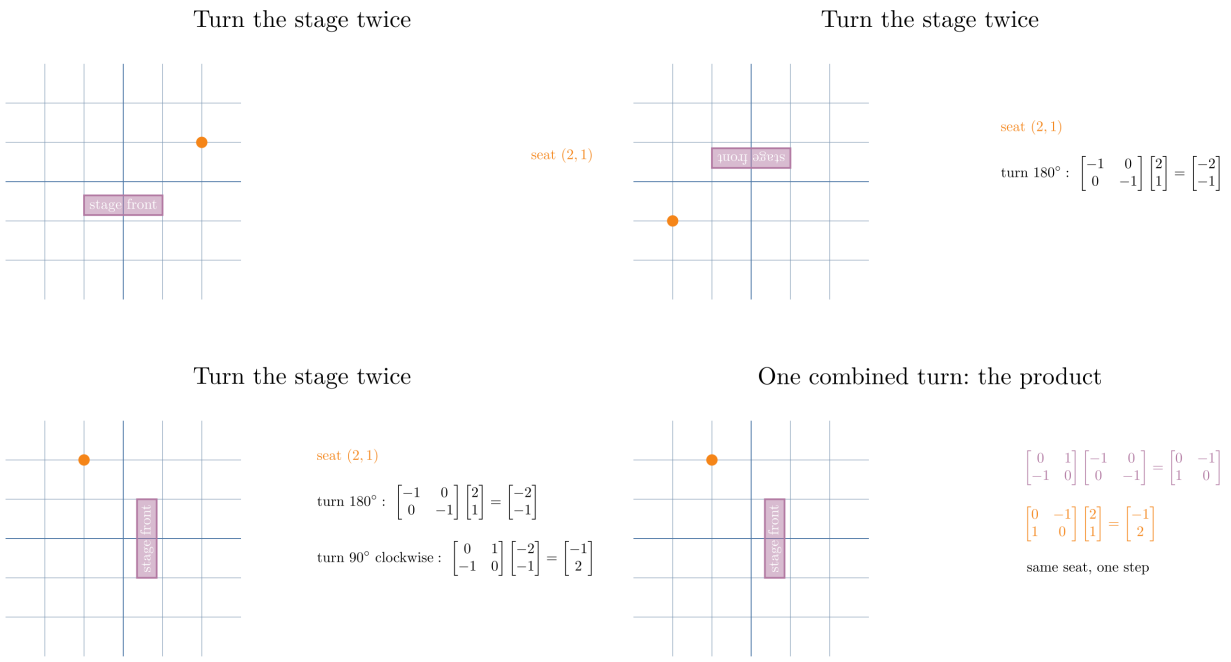


Figure 2: A seat on a turning stage: two turns move it from (2, 1) to (-2, -1) to (-1, 2); the product of the two turn matrices, a single 90° counterclockwise turn, sends it there in one step. Story after StatQuest, "Essential Matrix Algebra for Neural Networks"

In Figure 2, compare the last frame with the third: the combined turn lands the seat on the same spot, $(-1, 2)$, in one step. However many times the stage turns, multiplying the turn matrices gives one combined turn. That is the reason matrix multiplication is defined the way it is: the product is built so that it combines a sequence of transformations into one. Sections 2.2 to 4 show how to compute it.

2.2 Following the basis vectors

> **Key point:** To find the matrix of a composition, follow $\hat{\imath}$ and $\hat{\jmath}$ through both steps; where they finally land are its columns.

We often want to apply one transformation and then another. The overall effect is again a linear transformation (grid lines are still parallel and evenly spaced, and the origin has not moved). The overall effect is called the **composition** (G-431) of the two transformations.

Like any linear transformation, the composition has a matrix, found by following $\hat{\imath}$ and $\hat{\jmath}$ to their final places. In Figure 1, a rotation is followed by a shear:

- **Rotation by 90°** has the matrix R with columns $[0, 1]$ and $[-1, 0]$: it sends $\hat{\imath}$ to $[0, 1]$ and $\hat{\jmath}$ to $[-1, 0]$.
- **The shear** has the matrix S with columns $[1, 0]$ and $[1, 1]$: it keeps $[1, 0]$ fixed and sends $[0, 1]$ to $[1, 1]$.

- **After both**, $\hat{\imath}$ has ended at $[1, 1]$ and $\hat{\jmath}$ at $[-1, 0]$.

So the composition has the matrix with those two columns:

$$SR = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

Each entry is a row of S times a column of R, one per line:

$$\text{row 1, column 1} = (1)(0) + (1)(1) = 0 + 1 = 1$$

$$\text{row 1, column 2} = (1)(-1) + (1)(0) = -1 + 0 = -1$$

$$\text{row 2, column 1} = (0)(0) + (1)(1) = 0 + 1 = 1$$

$$\text{row 2, column 2} = (0)(-1) + (1)(0) = 0 + 0 = 0$$

$$SR = \begin{bmatrix} 1 & -1 \\ 1 & 0 \end{bmatrix}$$

The columns $[1, 1]$ and $[-1, 0]$ are where $\hat{\imath}$ and $\hat{\jmath}$ ended.

This one matrix does in a single step what the rotation and the shear do in two.

3. Why the product is written right to left

> **Key point:** In $SR\mathbf{x}$, the matrix next to the vector acts first: R is applied first, then S.

Applying the rotation and then the shear to a vector $\mathbf{x}$ the long way means two multiplications: first $R\mathbf{x}$, then S times that result, $S(R\mathbf{x})$. The **composition** (G-431) matrix must give the same answer for every $\mathbf{x}$, since it describes the same overall movement. So it is natural to call it the **product** (G-1179) SR:

$$S(R\mathbf{x}) = (SR)\,\mathbf{x}$$

Note the order. The transformation on the right (R) happens first and the one on the left (S) second.

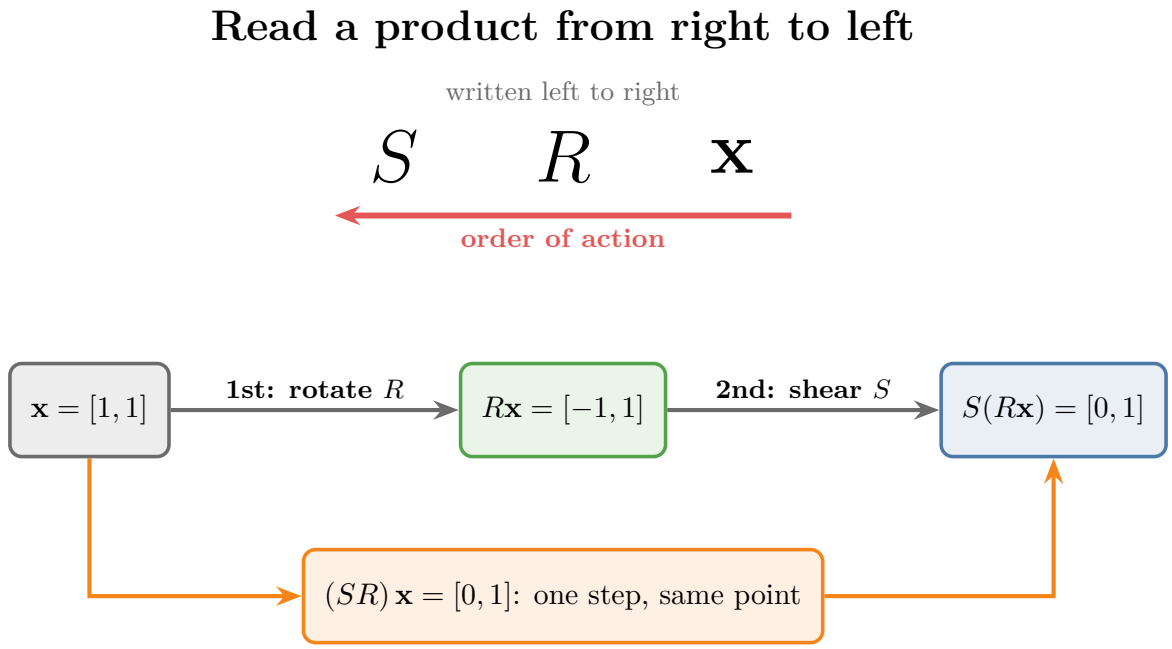


Figure 3: The product is written left to right but acts right to left; the vector [1, 1] goes through R, then S, and SR takes it to the same point in one step

In Figure 3, follow $[1, 1]$: R turns it into $[-1, 1]$, S then gives $[0, 1]$, and SR gives $[0, 1]$ in one step. The right-to-left order comes from function notation: in $f(g(x))$, g acts first. A matrix product is read from right to left.

4. Computing a product column by column

> **Key point:** The first column of M_2M_1 is M_2 times the first column of M_1; the second column is M_2 times the second column of M_1.

We can find the **product** (G-1179) without watching any animation, from the numbers alone. Take

$$M_1 = \begin{bmatrix} 1 & -2 \\ 1 & 0 \end{bmatrix}$$

$$M_2 = \begin{bmatrix} 0 & 2 \\ 1 & 0 \end{bmatrix}$$

and apply M_1 first, then M_2.

1. **In words:** after M_1, $\hat{\imath}$ sits at the first column of M_1. Applying M_2 to that column gives where $\hat{\imath}$ finally lands: the first column of the product. Do the same with the second column for $\hat{\jmath}$.

2. **Formula:**

$$\begin{aligned}&\text{column 1 of } M_2M_1\\ &= M_2 \cdot (\text{column 1 of } M_1)\\ &\text{column 2 of } M_2M_1\\ &= M_2 \cdot (\text{column 2 of } M_1)\end{aligned}$$

3. **Example:**

   - $\hat{\imath}$ first lands on $[1, 1]$. Then M_2 moves it:

$$M_2[1, 1] = 1 \cdot [0, 1] + 1 \cdot [2, 0]$$

$$M_2[1, 1] = [2, 1]$$

   - $\hat{\jmath}$ first lands on $[-2, 0]$. Then M_2 moves it:

$$M_2[-2, 0] = -2 \cdot [0, 1] + 0 \cdot [2, 0]$$

$$M_2[-2, 0] = [0, -2]$$

   - Check one entry, row 1 of M_2 times column 1 of M_1, the top-left entry below:

$$(0)(1) + (2)(1) = 0 + 2 = 2$$

   Put together, the two columns $[2, 1]$ and $[0, -2]$ are the product:

$$M_2M_1 = \begin{bmatrix} 2 & 0 \\ 1 & -2 \end{bmatrix}$$

Figure 4 runs this example on the grid. Watch $\hat{\imath}$ (green) and $\hat{\jmath}$ (red): after M_1 they sit on the columns of M_1, and after M_2 each one's final position fills one column of the product. Following the basis vectors like this is the approach of Sanderson's *Essence of Linear Algebra*, chapter 4 (3Blue1Brown).

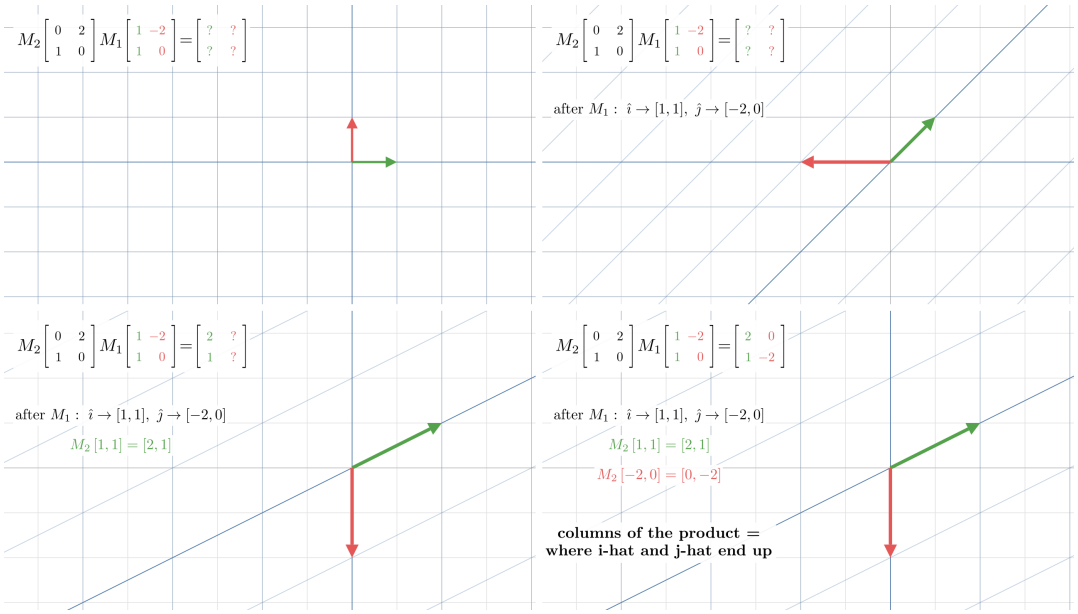


Figure 4: The product M_2M_1 built column by column: M_1 moves the basis vectors to its columns, M_2 then moves them to $[2,1]$ and $[0,-2]$, the columns of the product

The same reasoning with letters gives the general rule. Take

$$M_1 = \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

$$M_2 = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

The columns of M_2 are $[a,c]$ and $[b,d]$. The first column of M_1 is $[e,g]$, so M_2 sends it to e times the first column of M_2 plus g times the second; the same for the second column $[f,h]$:

$$\text{column } 1 = e\,[a,c] + g\,[b,d]$$

$$\text{column } 2 = f\,[a,c] + h\,[b,d]$$

Written as one matrix:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

$$= \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

Extra: The same formula read entry by entry is the school rule "row times column". The entry in row i, column j of AB is the dot product of row i of A with column j of B. In the example, row 1 of M_2 is $[0, 2]$ and column 1 of M_1 is $[1, 1]$. Their dot product is the top-left entry:

$$0 \times 1 + 2 \times 1 = 2$$

The row-times-column view is why the dot product is called the building block of matrix multiplication.

Python: `@` multiplies two matrices.

```python
import numpy as np

M1 = np.array([[1, -2], [1, 0]])
M2 = np.array([[0, 2], [1, 0]])
M2 @ M1          # [[2, 0], [1, -2]]
x = np.array([1, 1])
M2 @ (M1 @ x)    # [2, -1]: two steps
(M2 @ M1) @ x    # [2, -1]: one step
```

5. Order matters

Key point: In general $AB \neq BA$: shearing then rotating is a different transformation from rotating then shearing.

Does the order of the two matrices matter? Thinking in transformations, we can answer by picturing them. Figure 5 runs the rotation R and the shear S of Section 2.2 in both orders at once; watch the two pairs of arrows end up in different places.

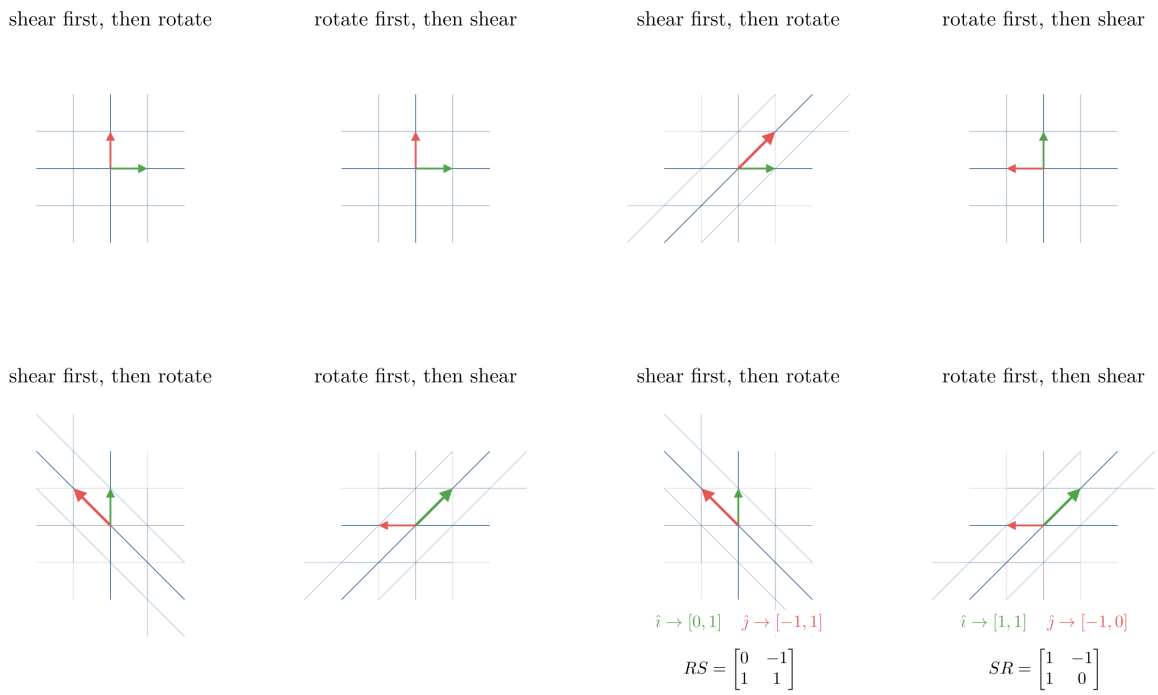


Figure 5: The same two transformations in the two possible orders, side by side; at the end, compare where $\hat{\imath}$ (green) and $\hat{\jmath}$ (red) have landed. Example after 3Blue1Brown, "Matrix multiplication as composition"

- **Shear first, then rotate** (RS): $\hat{\imath}$ ends at $[0, 1]$ and $\hat{\jmath}$ at $[-1, 1]$. The two vectors point close together.
- **Rotate first, then shear** (SR): $\hat{\imath}$ ends at $[1, 1]$ and $\hat{\jmath}$ at $[-1, 0]$. They point far apart.

The matrix SR was computed in section 2.2. The matrix RS, entry by entry (row of R times column of S):

$$\text{row 1, column 1} = (0)(1) + (-1)(0) = 0$$

$$\text{row 1, column 2} = (0)(1) + (-1)(1) = -1$$

$$\text{row 2, column 1} = (1)(1) + (0)(0) = 1$$

$$\text{row 2, column 2} = (1)(1) + (0)(1) = 1$$

The overall effects differ, so $RS \neq SR$:

$$RS = \begin{bmatrix} 0 & -1 \\ 1 & 1 \end{bmatrix}$$

$$SR = \begin{bmatrix} 1 & -1 \\ 1 & 0 \end{bmatrix}$$

Matrix multiplication is **not commutative** (G-1352), unlike the multiplication of numbers. In code this means `A @ B` and `B @ A` are, in general, different matrices.

6. Grouping does not matter

> **Key point:** $(AB)C = A(BC)$, because both mean "apply C, then B, then A".

For three matrices A, B and C, we can first compute AB and multiply the result by C, or first compute BC and put A in front. Both give the same matrix:

$$(AB)\,C = A\,(BC)$$

This property is called **associativity** (G-219). Proving it by expanding all the entries is long and tells us nothing. Seen as transformations it is immediate: both sides mean "apply C, then B, then A", the same three steps in the same order. Only the bookkeeping differs.

> **Python:** A quick check with the three matrices of this Note.

```
R = np.array([[0, -1], [1, 0]])
S = np.array([[1, 1], [0, 1]])
np.array_equal((S @ R) @ M1, S @ (R @ M1))    # True
np.array_equal(S @ R, R @ S)                  # False
```

7. Where ML multiplies matrices

> **Key point:** A **product** (G-1179) needs matching inner sizes; stacked linear layers collapse into one matrix, which is why neural networks need activation functions between them.

7.1 The shape rule

> **Key point:** An $m \times n$ matrix times an $n \times p$ matrix gives an $m \times p$ matrix; the inner sizes must match.

So far all matrices were 2×2. In ML the matrices are rectangular, and the column-by-column view explains the shape rule. In AB, each column of B is a vector that A transforms, so each column of B must have as many entries as A has columns.

1. **In words:** the number of columns of the left matrix must equal the number of rows of the right matrix; the result takes the rows of the left and the columns of the right.
2. **Formula:**

$$(m \times n)\,(n \times p) = (m \times p)$$

3. **Example:** the data matrix X with 3 points and 2 **features** (G-772) (input variables, one column each) times A^{T} (the **transpose** (G-2012) of A, see section 7.2) in one matrix for the whole dataset gives three transformed points:

$$(3 \times 2)(2 \times 2) = 3 \times 2$$

PCA's XW^{T} with 40 points, 3 features and 2 components gives:

$$(40 \times 3)(3 \times 2) = 40 \times 2$$

(see the projection step of PCA).

A **product** (G-1179) with mismatched inner sizes does not exist. The turn matrix T_1 of Section 2.1 times the seat written as a row fails:

$$(2 \times 2)(1 \times 2)$$

Each row of T_1 has two numbers, but each column of $[2 \;\; 1]$ has only one to pair them with. The dot product $a^{\mathsf{T}}b$ is the smallest case of the rule:

$$(1 \times n)(n \times 1) = 1 \times 1$$

7.2 Points as rows: the same product, transposed

> **Key point:** Libraries that store points as rows write $\mathbf{x}^{\mathsf{T}}W^{\mathsf{T}}$ instead of $W\mathbf{x}$; then the matrix applied first stands on the left.

This Note writes a point as a column and puts the matrix on its left, $W\mathbf{x}$. A data table stores each point as a row, so many libraries write the same step with the point on the left. Turning rows into columns is the **transpose** (G-2012); it flips a matrix across its diagonal, so the rows of W become the columns of W^{T}.

1. **In words:** in **row form** (G-2245), each point is a 1×2 row, and it is multiplied on the right by the transposed matrix.
2. **Example:** the stage of Section 2.1, with the seat as the row $[2 \;\; 1]$. Here $T_1^{\mathsf{T}} = T_1$, so the first turn gives:

$$[2 \;\; 1]\, T_1^{\mathsf{T}} = [-2 \;\; -1]$$

The second turn uses T_2^{T}, whose columns are $[0, 1]$ and $[-1, 0]$. Row times first column:

$$-2 \times 0 + (-1) \times 1 = -1$$

Row times second column:

$$-2 \times (-1) + (-1) \times 0 = 2$$

The seat lands on $[-1 \;\; 2]$, as before.

3. **Formula:** the two forms are transposes of each other,

$$(T_2 T_1 \mathbf{x})^\mathsf{T} = \mathbf{x}^\mathsf{T} T_1^\mathsf{T} T_2^\mathsf{T}$$

   In row form the matrix applied first, T_1^T, stands next to the point on the left, and the **product** (G-1179) reads left to right.

PyTorch's linear layer uses the row form: `torch.nn.Linear` computes $y = xA^\mathsf{T} + b$, with the input x as a row and the weights A stored one neuron per row (PyTorch docs, `torch.nn.Linear`). Both forms give the same numbers; only the bookkeeping differs.

Python: the two forms agree.

```
T1 = np.array([[-1, 0], [0, -1]])
T2 = np.array([[0, 1], [-1, 0]])
T2 @ T1 @ np.array([2, 1])          # [-1, 2]: columns
np.array([[2, 1]]) @ T1.T @ T2.T    # [[-1, 2]]: rows
```

7.3 Linear layers collapse into one

Key point: Two layers without an activation, $W_2(W_1\mathbf{x})$, are the single layer $(W_2W_1)\mathbf{x}$: stacking them adds no power.

A neural network applies layer after layer. Suppose two layers had no activation function between them, only the matrices W_1 and then W_2. By Section 3, the two steps are one transformation:

$$W_2(W_1\mathbf{x}) = (W_2W_1)\,\mathbf{x}$$

With the numbers of Section 4, take $W_1 = M_1$ and $W_2 = M_2$, and the input $[1, 1]$. Layer by layer:

$$M_1[1, 1] = [-1, 1]$$

$$M_2[-1, 1] = [2, -1]$$

In one step, with the **product** (G-1179) matrix:

$$(M_2M_1)[1, 1] = [2, -1]$$

However many linear layers we stack, the result is still one matrix, one linear transformation.

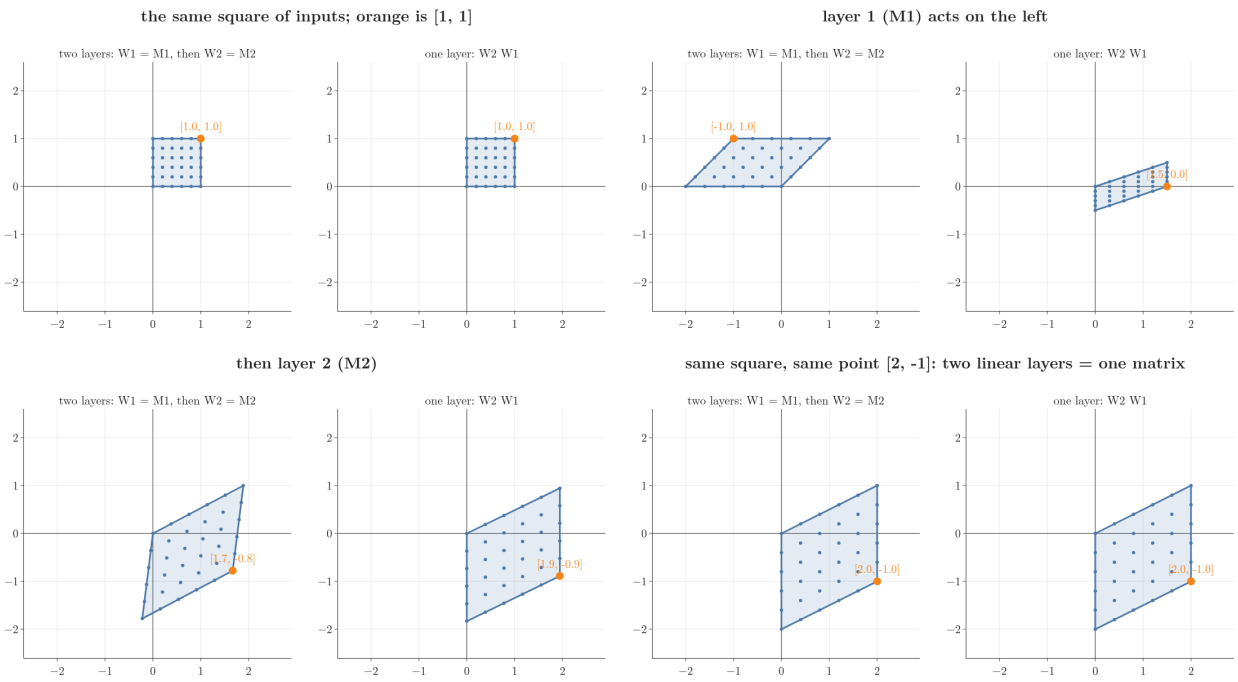


Figure 6: Two linear layers (left) and their single product matrix (right) move a square of inputs to the same place

In Figure 6, compare the two panels at the end: the same parallelogram, and $[1, 1]$ at $[2, -1]$ in both.

The collapse is why the hidden layers of a network end with a non-linear activation such as the sigmoid. The activation bends the space between layers, so the **composition** (G-431) can no longer be squeezed into a single matrix. The classic example is XOR (the output is 1 only when exactly one of the two inputs is 1): no linear model can fit it, while a network with one hidden layer and a non-linear activation can (Goodfellow et al. §6.1).

> **Extra:** The same collapse shows in linear regression with engineered features. If every new feature is a **linear combination** (G-1091) of the old ones, the model can learn nothing new: the combined effect is still a linear combination of the original features. Polynomial features (see **polynomial regression** (G-1515)) help precisely because squaring is not linear.

8. Summary

Idea	What it says	Example	Why it matters
Composition	one transformation after another	rotate, then shear	it is still linear, so it has one matrix
Product BA	matrix of "apply A, then B"	$SR = \begin{bmatrix} 1 & -1 \\ 1 & 0 \end{bmatrix}$	does in one step what A then B do in two
Column by column	column j of BA is B times column j of A	$M_2[1,1] = [2,1]$	gives the product from the numbers alone
Not commutative	$AB \neq BA$ in general	$RS \neq SR$	`A @ B` and `B @ A` are different matrices in code
Associative	$(AB)C = A(BC)$	same three steps, same order	a long product can be bracketed either way
Transpose	turning the rows of a matrix into its columns	$(AB)^\mathsf{T} = B^\mathsf{T}A^\mathsf{T}$	switches between points as columns and points as rows
Row form	points as rows, multiplied by W^T on the right	$\mathbf{x}^\mathsf{T}W^\mathsf{T}$: the matrix applied first sits on the left	data tables and PyTorch's linear layer store points this way
Shape rule	$(m\times n)(n\times p) = m\times p$	$(40\times 3)(3\times 2) = 40\times 2$	a product with mismatched inner sizes does not exist

- A **matrix product** (G-1179) is a composition of transformations, read from right to left, because the matrix next to the vector acts first, as g does in $f(g(x))$.
- Follow $\hat{\imath}$ and $\hat{\jmath}$ through both steps to get the columns of the product, because the columns of any matrix are where the basis vectors land.
- Order matters, because shearing then rotating is a different movement from rotating then shearing; grouping does not, because both groupings apply the same steps in the same order.
- Stacked linear layers collapse into one matrix; activation functions prevent that, so a network can fit patterns such as XOR that no linear model can.
- So multiplying two matrices gives the one matrix that does both transformations, the right-hand one first.

9. Sources

Built from

- Starmer, J. (StatQuest), "Essential Matrix Algebra for Neural Networks, Clearly

Explained!!!", YouTube, https://www.youtube.com/watch?v=ZTt9gsGcdDo
- Sanderson, G. (3Blue1Brown), "Matrix multiplication as composition | Chapter 4, Essence of linear algebra", 2016, 3blue1brown.com/lessons/matrix-multiplication, https://www.youtube.com/watch?v=XkY2DOUCWMU

Other references

- PyTorch documentation, `torch.nn.Linear`, pytorch.org/docs/stable/generated/torch.nn.Linear.html.
- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press. Section 6.1, learning XOR.

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Matrix product (G-1179)	The single matrix BA that does the same as applying A and then B (their composition), so a chain of transformations can be done as one multiplication.
Composition (G-431)	Applying one transformation and then another, seen as one overall transformation.
Not commutative (G-1352)	The order of the factors matters: $AB \neq BA$ in general.
Associativity (G-219)	A property of matrix multiplication: in a product of three matrices, which pair is multiplied first does not change the result, because both groupings mean apply C, then B, then A; written $(AB)C = A(BC)$.
Row form (G-2245)	Writing points as rows and multiplying $\mathbf{x}^\mathsf{T}W^\mathsf{T}$; the matrix applied first is on the left.
Shape rule (G-1786)	The rule for which matrices can be multiplied: an $m \times n$ matrix times an $n \times p$ matrix gives an $m \times p$ matrix, so the inner sizes must match.
Linear combination (G-1091)	A sum of vectors each multiplied by a number, $a_1\mathbf{v}_1 + \cdots + a_k\mathbf{v}_k$; changing the numbers reaches different vectors, and all the vectors it can reach form the span.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Transpose (G-2012)	A matrix or vector with rows and columns swapped; it turns a column vector u into a row u^T, so $u^\mathsf{T}x$ is the dot product.

Term	Meaning
Polynomial regression (G-1515)	Linear regression on powers (and products) of the inputs, to fit curves.